



由表4-1可见，自然顺序数 I 增加1，是在顺序数的二进制数最低位加1，向左进位。而倒序数 J 则是在二进制数最高位加1，逢2向右进位。例如，在(000)最高位加1，则得(100)，再在(100)最高位加1，向右进位，则得(010)。因(100)最高位为1，所以最高位加1要向次高位进位，其实质是将最高位变为0，再在次高位加1。用这种算法，可以从当前任一倒序值求得下一个倒序值。



对于 $N=2^M$ ， M 为二进制数最高位，其权值为 $N/2$ ，且从左向右二进制位的权值依次为 $N/4$ ， $N/8$ ， $\dots$ ， 2 ， 1 。因此，最高位加1相当于十进制运算 $J+N/2$ 。如果最高位是0（ $J < N/2$ ），则直接由 $J+N/2$ 得下一个倒序值；如果最高位是1（ $J \geq N/2$ ），则要将最高位变为0（ $J \leftarrow J - N/2$ ），次高位加1（ $J+N/4$ ）。但次高位加1时，同样要判断次高位0、1值，如果为0（ $J < N/4$ ），则直接加1（ $J \leftarrow J + N/4$ ）；否则，将次高位变为0（ $J \leftarrow J - N/4$ ），再判断下一位；以此类推，直到完成最高位加1，逢2向右进位的运算。



把按自然顺序存放在存储单元中的数据，换成FFT原位运算流图所要求的倒位序的变址功能如图4-8所示，当 $I=J$ 时，不必调换，当 $I \neq J$ 时，必须将原来存放数据 $x(I)$ 的存储单元内调入数据 $x(J)$ ，而将存放 $x(J)$ 的存储单元内调入 $x(I)$ 。为了避免把已调换过的数据再次调换，保证只调换一次（否则又回到原状），我们只需看 J 是否比 I 小。若 J 比 I 小，则意味着此 $x(I)$ 在前边已和 $x(J)$ 互相调换过，不必再调换了；只有当 $J > I$ 时，才将原存放 $x(I)$ 及存放 $x(J)$ 的存储单元内的内容互换。这样就得到输入所需的倒位序列的顺序。可以看出，其结果与图4-8的要求是一致的。

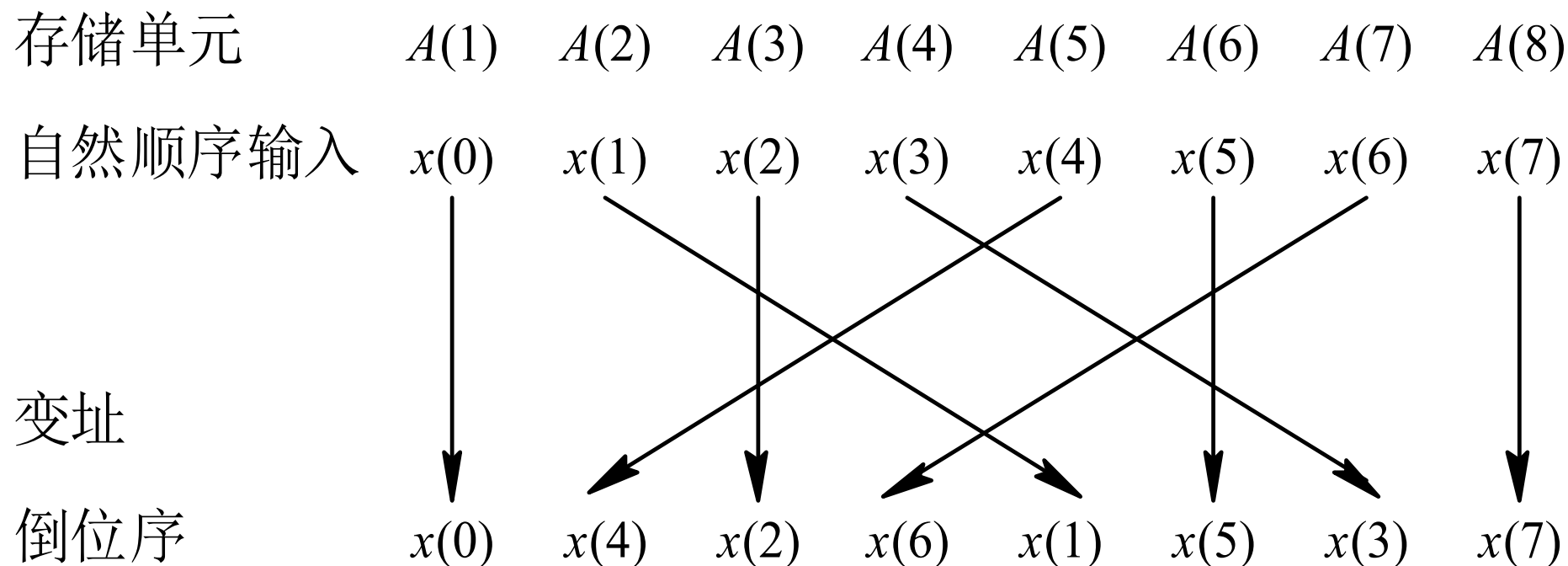


图 4-8 $N=8$ 倒位序的变址处理



§4.3 按频率抽取的FFT

频率抽取法是 $N=2^M$ 情况下的另外一种FFT算法

按时间抽取FFT
算法的基本思路

$\left\{ \begin{array}{l} x(n) \text{按奇、偶一级级分开,} \\ X(k) \text{按前一半、后一半一级级分开。} \end{array} \right.$

按频率抽取FFT
算法的基本思路

$\left\{ \begin{array}{l} X(k) \text{按奇、偶一级级分开,} \\ x(n) \text{按前后对半一次次分开。} \end{array} \right.$

一.按频率抽取的FFT算法原理

$$X(k) = \sum_{n=0}^{N-1} x(n) W_N^{nk}$$

将 $x(n)$ 按 n 的顺序
前后对半分开



$$X(k) = \sum_{n=0}^{N/2-1} x(n) W_N^{nk} + \sum_{n=N/2}^{N-1} x(n) W_N^{nk} \quad \text{令 } n' = n - \frac{N}{2}$$

$$= \sum_{n=0}^{N/2-1} x(n) W_N^{nk} + \sum_{n'=0}^{N/2-1} x(n' + \frac{N}{2}) W_N^{(n' + \frac{N}{2})k}$$

$$= \sum_{n=0}^{N/2-1} [x(n) + W_N^{(N/2)k} x(n + \frac{N}{2})] W_N^{nk}$$

$$\because W_N^{N/2} = -1, \quad = (-1)^k = \begin{cases} 1, & k \text{ 为偶数} \\ -1, & k \text{ 为奇数} \end{cases}$$

► 按 k 的奇、偶把 $X(k)$ 进一步分解为两部分：



$$X(k) = \sum_{n=0}^{N/2-1} [x(n) + (-1)^k x(n + N/2)] W_N^{nk}$$

a) k 为偶数, 令 $k = 2r$ $r = 0, 1, \dots, \frac{N}{2} - 1$

$$X(2r) = \sum_{n=0}^{N/2-1} [x(n) + x(n + N/2)] W_N^{2nr}$$

$$\because W_N^{2nr} = e^{-j \frac{2\pi}{N/2} nr} = W_{N/2}^{nr}$$

$$= \sum_{n=0}^{N/2-1} \underbrace{[x(n) + x(n + N/2)]}_{x_1(n)} W_{N/2}^{nr}$$

$$X(2r) = \sum_{n=0}^{N/2-1} x_1(n) W_{N/2}^{nr} = \text{DFT}[x_1(n)]_{N/2}$$



b) k 为奇数, 令 $k=2r+1$

$$w_N^{2nr} = e^{-j \frac{2\pi}{N/2} nr} = w_{N/2}^{nr}$$

$$X(2r+1) = \sum_{n=0}^{N/2-1} [x(n) - x(n+N/2)] W_N^{n(2r+1)}$$

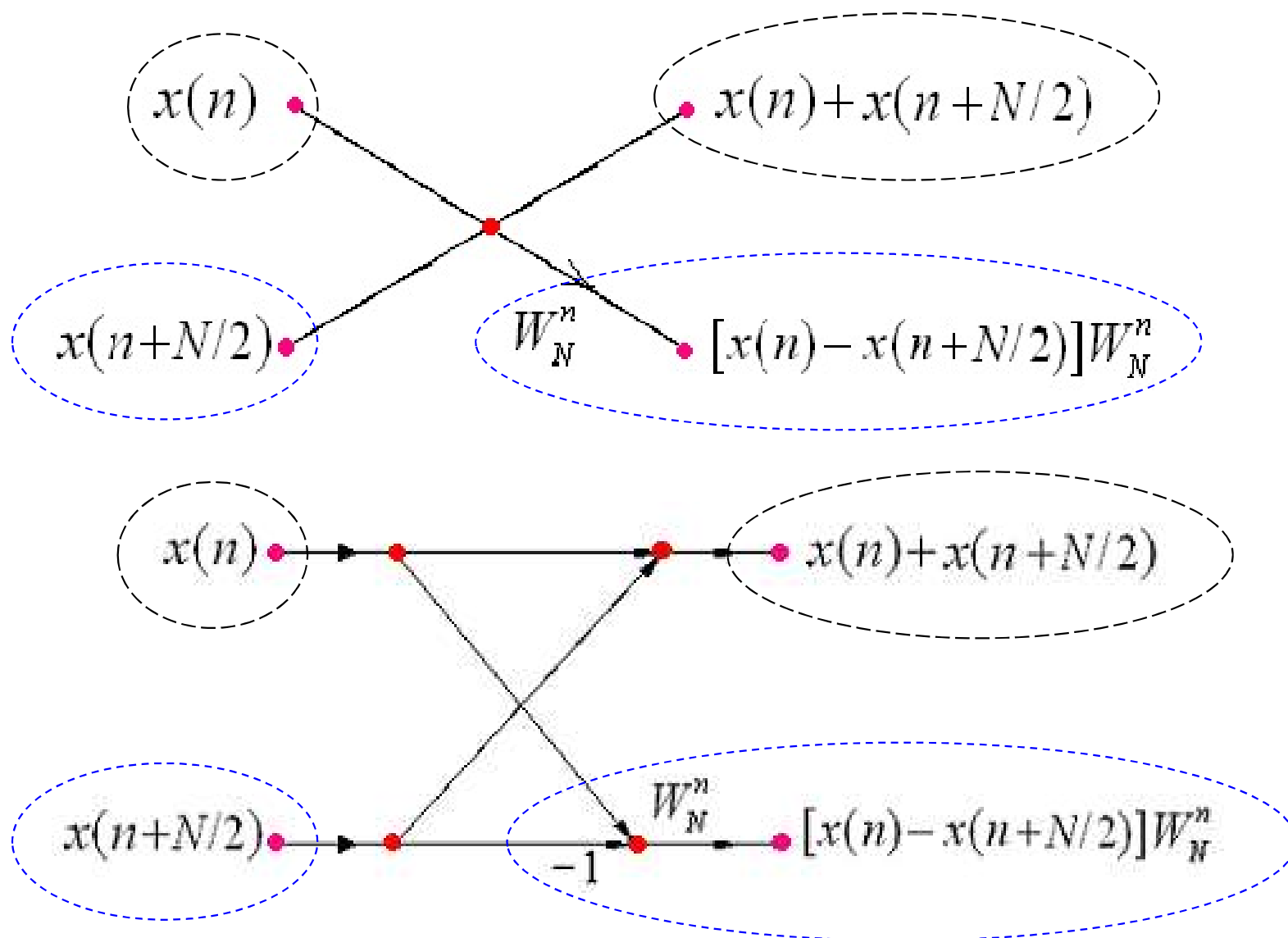
$$= \sum_{n=0}^{N/2-1} \underbrace{[x(n) - x(n+N/2)] W_N^n}_{x_2(n)} W_{N/2}^{nr}$$

$$= \sum_{n=0}^{N/2-1} x_2(n) W_{N/2}^{nr} = \text{DFT}[x_2(n)]_{N/2}$$

$$\left\{ \begin{array}{l} X(2r) = \sum_{n=0}^{N/2-1} [x(n) + x(n+N/2)] W_{N/2}^{nr} = \sum_{n=0}^{N/2-1} x_1(n) W_{N/2}^{nr} \\ X(2r+1) = \sum_{n=0}^{N/2-1} \{ [x(n) - x(n+N/2)] W_N^n \} W_{N/2}^{nr} = \sum_{n=0}^{N/2-1} x_2(n) W_{N/2}^{nr} \end{array} \right.$$

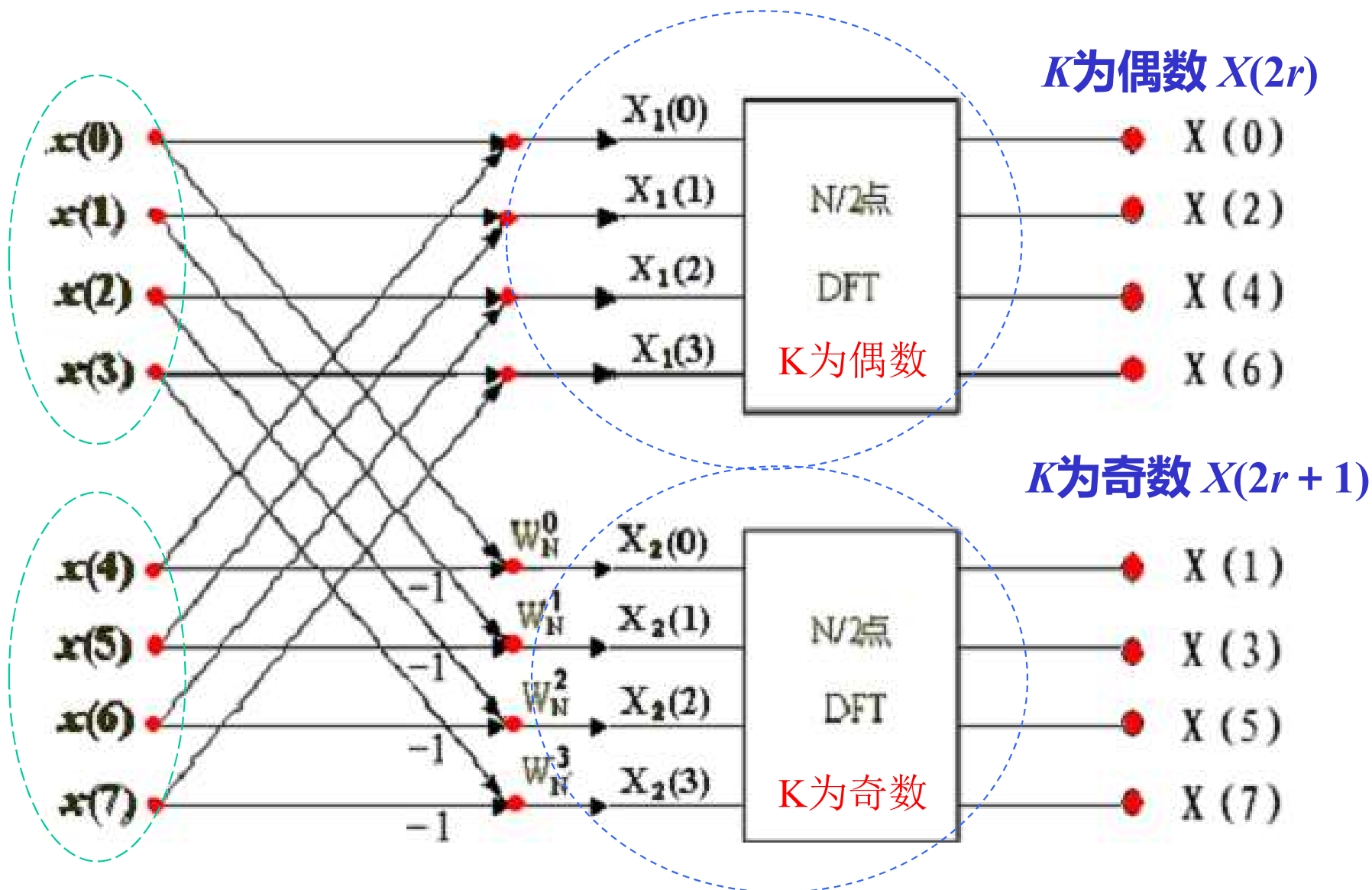


按频率抽取的蝶形运算流图：





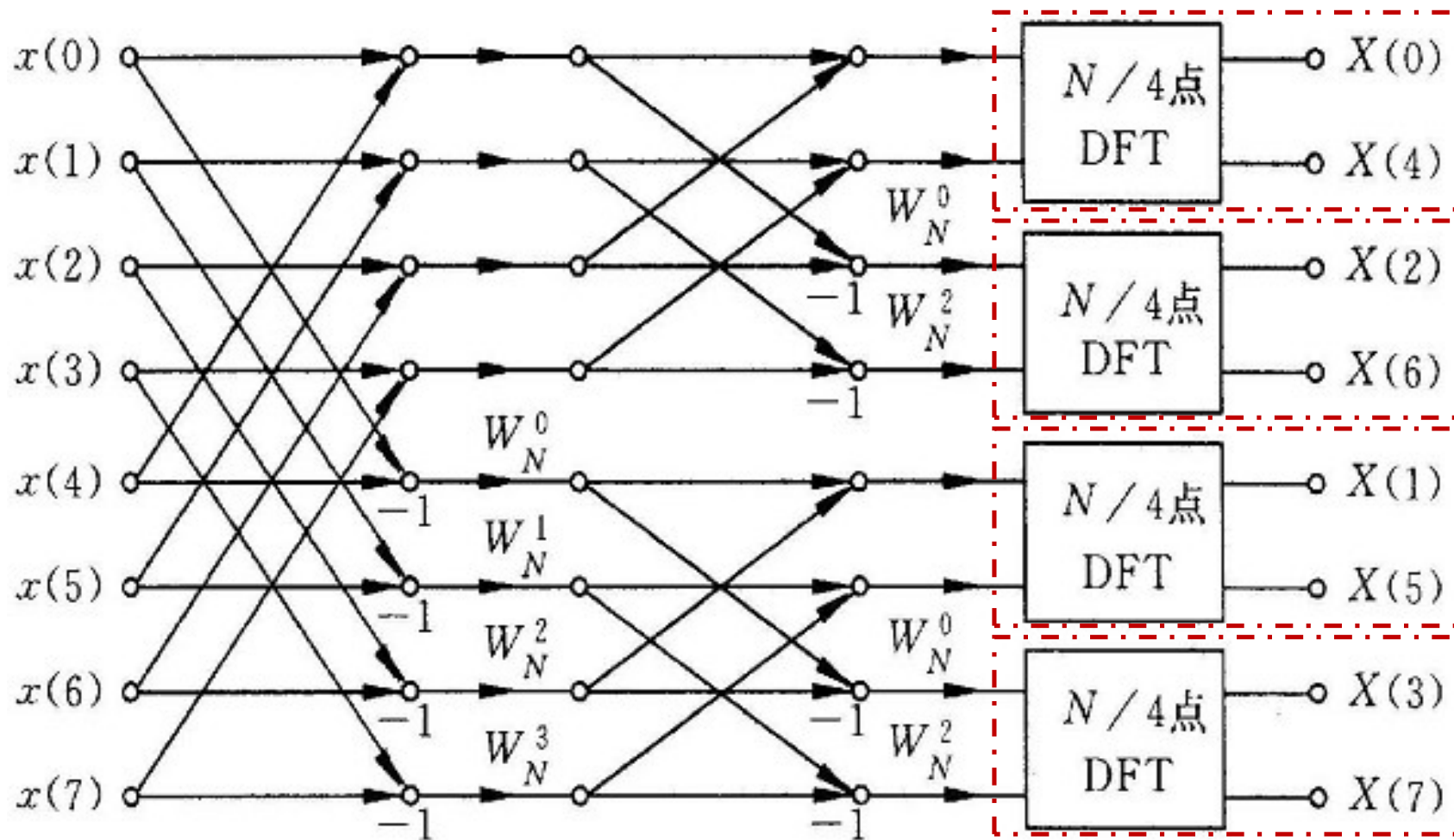
例：以 $N = 2^M = 2^3 = 8$ 按频率抽取的FFT流图



把8点DFT分解成两个4点的DFT



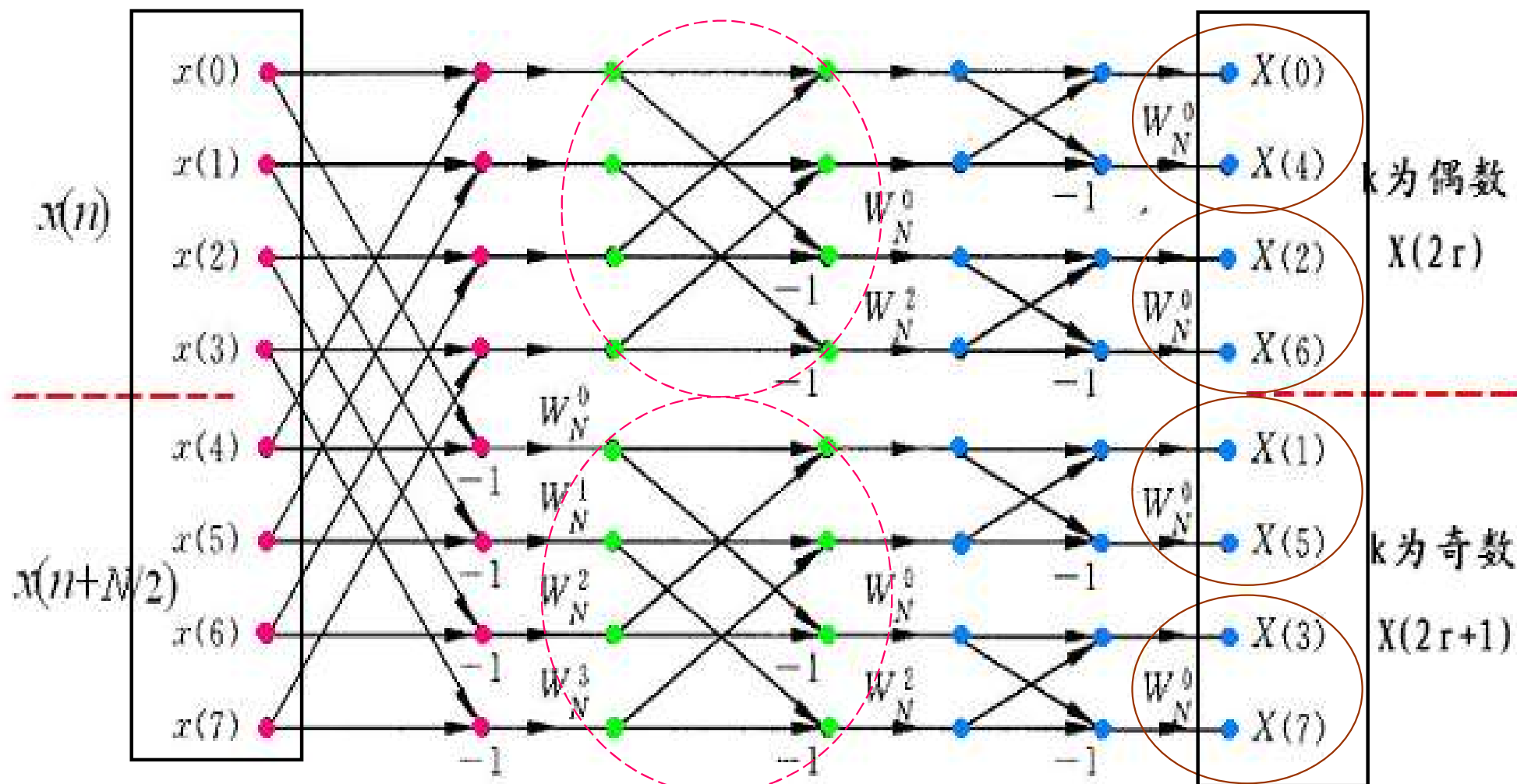
由于 $N/2$ 仍是偶数可以继续分解, 再按 k 值的奇、偶分解, 直到分解为两两点的DFT为止。



将两个4点DFT再分解成四个2点DFT



$N=2^3=8$ 按频率抽取的FFT的流图



- 分解过程：
- (1) 把输入序列前后对半分；
 - (2) 输出按 k 值的奇、偶将 $X(k)$ 分解为奇数组和偶数组；
 - (3) 输入如果按顺序输入，输出则是码位倒置形式。



按频率抽取法FFT的运算特点:

(1) 蝶形运算的运算量

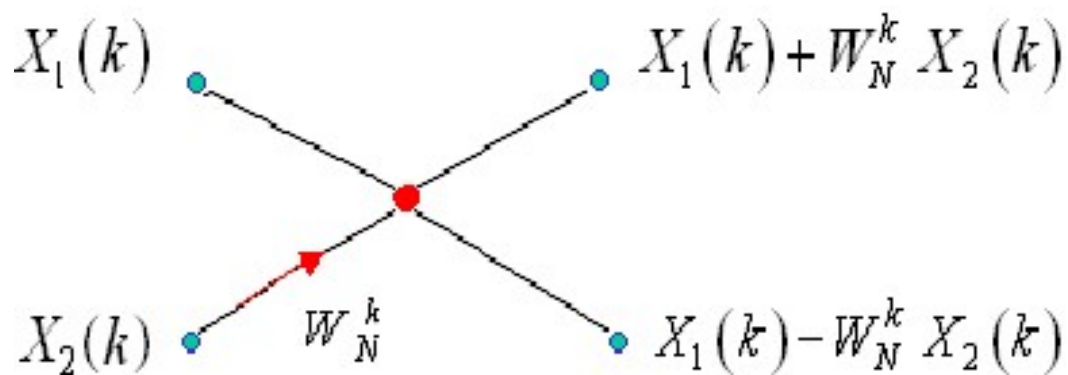
其运算量与按时间抽取运算量相同。即:

$$\text{复乘: } M_F = \frac{N}{2} \log_2 N \quad \text{复加: } a_F = N \log_2 N$$

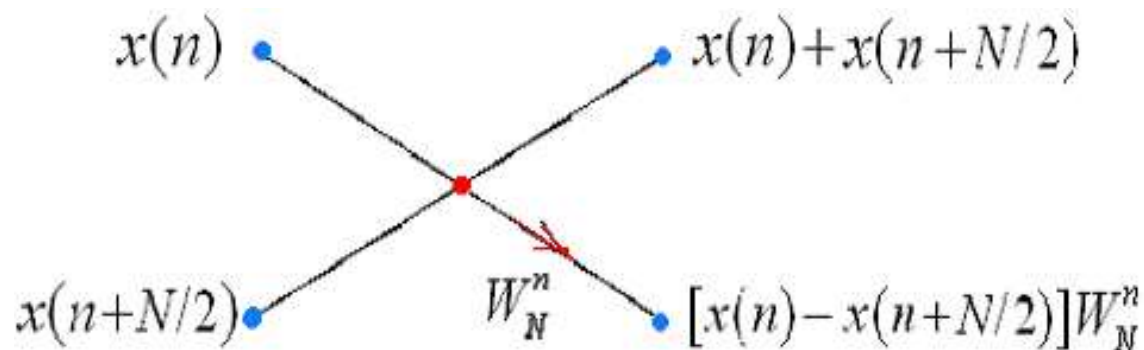
(2) 输入如果是自然顺序, 输出则是码位倒置形式。如果要求输出为自然顺序, 则需要进行变址运算。



(3) 频率抽取法的基本蝶形图与时间抽取法的基本蝶形图有所不同。时间抽取法是先乘后加、减，频率抽取法是先加、减，再乘系数 W_N^n 。



按时间抽取的
蝶形运算流图



按频率抽取的
蝶形运算流图



§ 4.4 IDFT的快速算法IFFT

按时间抽取的FFT算法以及按频率抽取的FFT算法是DFT的两种快速算法，这些算法同样可用于IDFT，我们称为快速傅立叶反变换(IFFT)。把IDFT与DFT公式作比较：

$$\left\{ \begin{array}{l} x(n) = \text{IDFT}[X(k)] = \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk} \quad 0 \leq n \leq N-1 \\ X(k) = \text{DFT}[x(n)] = \sum_{n=0}^{N-1} x(n) W_N^{nk} \quad 0 \leq k \leq N-1 \end{array} \right.$$

(1) DFT中的系数 $W_N^{nk} \Rightarrow W_N^{-nk}$

(2) IDFT中要乘以常数 $\frac{1}{N} = \left(\frac{1}{2}\right)^M$



以上讨论的按时间抽取或按频率抽取的FFT运算均可直接用于IFFT运算。但需要将蝶形系数 W_N^{-nk} 改为 W_N^{nk} ，每级乘以系数 $1/2$ 。

➤ 另外一种IFFT算法的实现

$$\begin{aligned} x(n) &= \frac{1}{N} \sum_{k=0}^{N-1} X(k) W_N^{-nk} = \frac{1}{N} \left[\sum_{k=0}^{N-1} X^*(k) W_N^{nk} \right]^* \\ &= \frac{1}{N} \left\{ \text{FFT} \left[X^*(k) \right] \right\}^* \end{aligned}$$



按上述方法求得的IFFT，完全不需要改动FFT程序，FFT和IFFT可以共用一个程序。

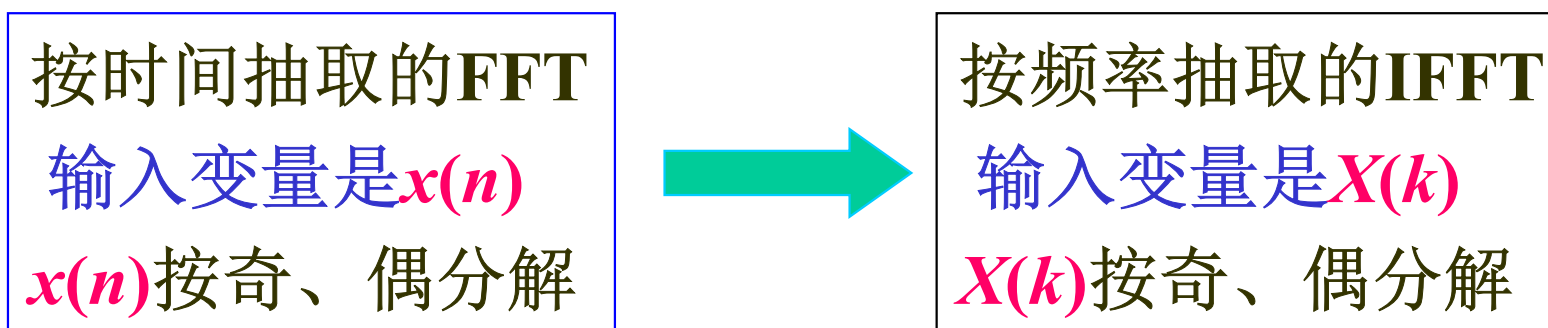
IFFT计算分为三步：

- ① 将 $X(k)$ 取共轭(虚部乘以-1) $X^*(k)$ ；
- ② 对 $X^*(k)$ 直接进行FFT；
- ③ 对FFT的结果取共轭并乘以 $1/N$ ，得到 $x(n)$ 。



➤ 总结FFT和IFFT蝶形图的表示方法:

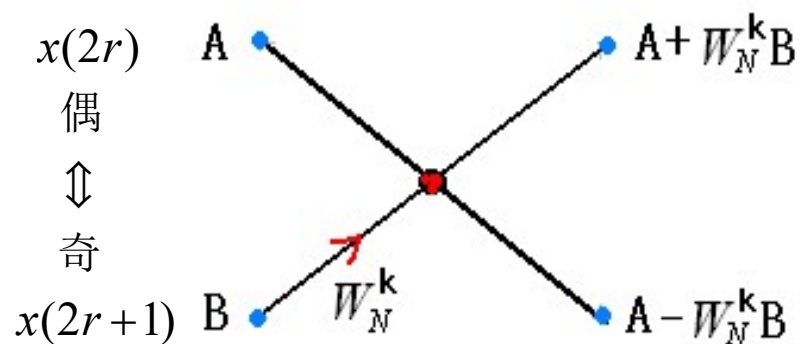
把按时间抽取的FFT $\longrightarrow$ 变到IFFT时, 对应的蝶形图应该是按频率抽取的IFFT。因为:



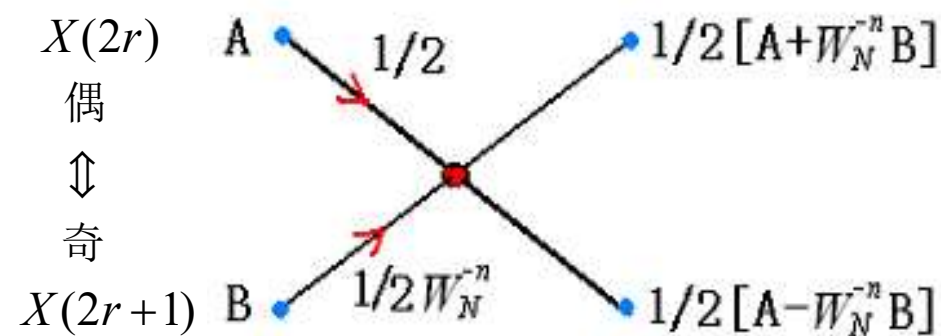
同理, 按频率抽取的FFT $\longrightarrow$ 变到IFFT时, 对应的蝶形图应该是按时间抽取的IFFT。



(1) 输入按奇、偶分解:

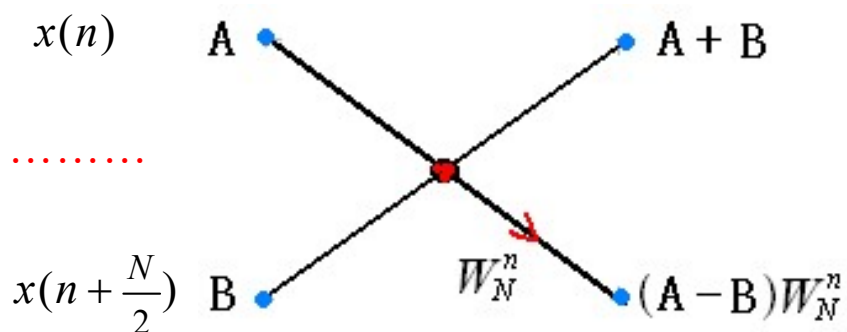


FFT时间抽取蝶形图

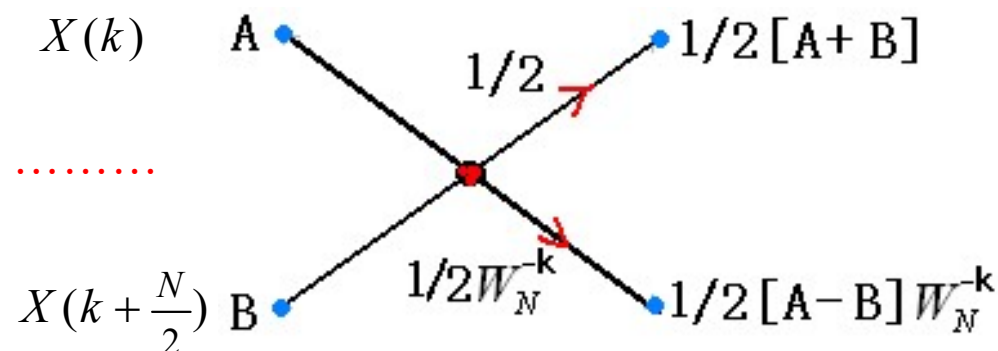


IFFT频率抽取蝶形图

(2) 输入按前、后分解:



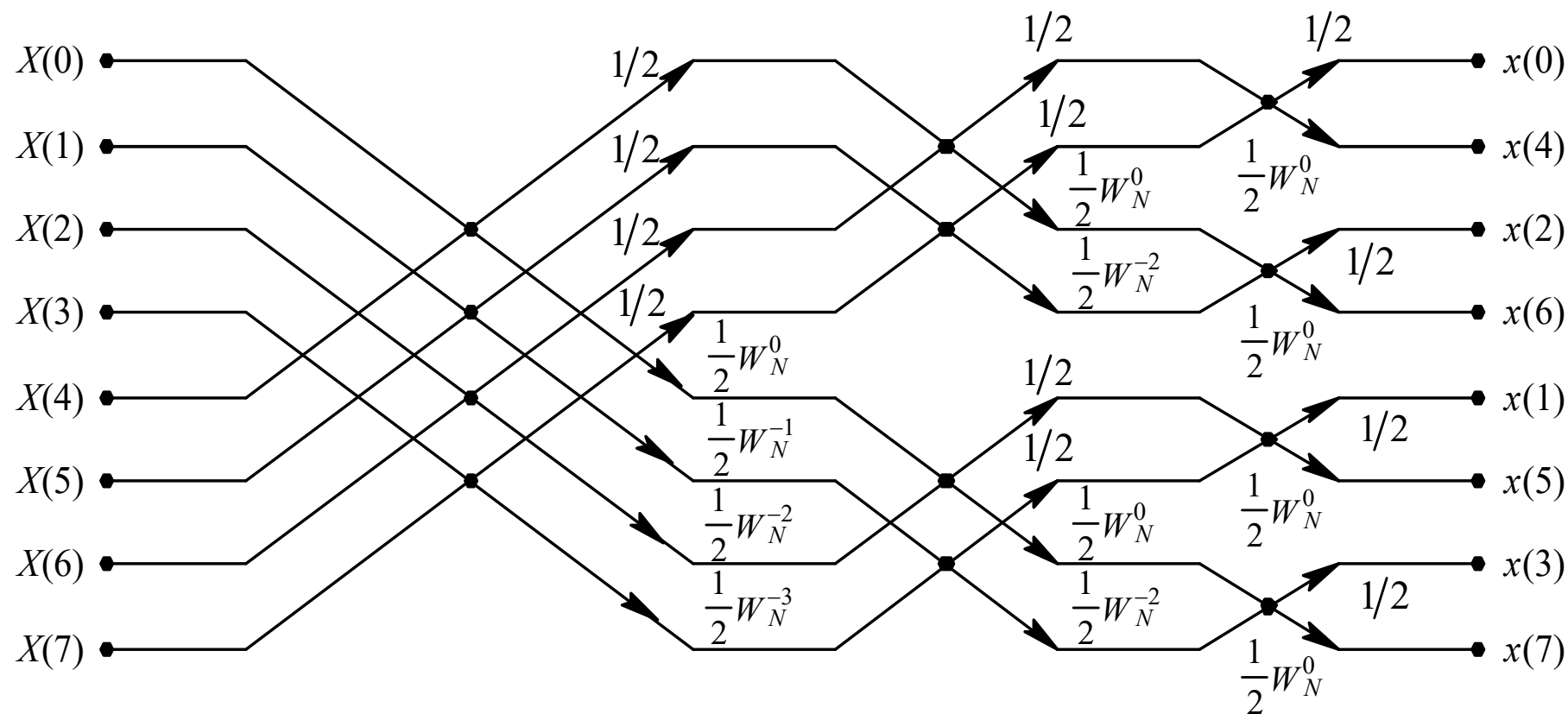
FFT频率抽取蝶形图



IFFT时间抽取蝶形图



$N=8$ 时间抽取IFFT流图:





§ 4.5 任意基数的FFT算法

前面讨论的FFT算法,序列的长度均满足 $N=2^M$, 因为它程序简单、高效、使用方便。如果不满足 $N=2^M$, 处理方法有两种:

① **补零**: 将 $x(n)$ 补零,使 N 增加到临近的一个值, 满足 $N=2^M$ 。

例如: $N=30$, 补上 $x(30)=x(31)=0$ 两个零点, 使得满足 $N=2^5=32$, 这样可直接调用以2为基数的FFT 运算。有限长序列补零后并不影响其频谱 $X(e^{j\omega})$, 补零只是频谱的采样点数增加了。



② 如 $N = 2052$, $N = 2^{12} = 4096$ 如果要求 N 点DFT值,几乎要补一半0值(2044个0), 显然采用第一种方法处理很不经济, 因此, 如果要求求准确的 N 点DFT值,可采用任意基数的FFT算法,但其运算效率低于以2为基数的FFT算法。

任意基数的FFT算法的基本思想

前提是 N 可以分解成因子乘积的形式, N 不能是素数。任意基数的FFT算法的基本思想仍是把一个大 N 点的FFT尽量分解为小点数的FFT。

首先将 N 分解为两个整数 p 与 q 的乘积,即: $N=p \cdot q$,
其步骤为:



(1)将序列 $x(n)$ 分成 p 组

$$p\text{组} \left\{ \begin{array}{l} x(p r) \\ x(p r + 1) \\ x(p r + 2) \\ \vdots \\ x(p r + p - 1) \end{array} \right. \quad r=0,1,\dots,q-1$$

(2)将 N 点DFT分成 p 组 q 点的DFT来运算

$$\begin{aligned} X(k) &= \sum_{n=0}^{N-1} x(n) W_N^{kn} \\ &= \sum_{r=0}^{q-1} x(pr) W_N^{prk} + \sum_{r=0}^{q-1} x(pr+1) W_N^{(pr+1)k} + \dots \\ &\quad + \sum_{r=0}^{q-1} x(pr+p-1) W_N^{(pr+p-1)k} \end{aligned}$$



$$= W_N^0 \sum_{r=0}^{q-1} x(pr) W_N^{prk} + W_N^k \sum_{r=0}^{q-1} x(pr+1) W_N^{prk} + \dots$$

$$+ W_N^{(p-1)k} \sum_{r=0}^{q-1} x(pr+p-1) W_N^{prk}$$

$$= \sum_{L=0}^{p-1} W_N^{Lk} \sum_{r=0}^{q-1} x(pr+L) W_N^{prk} \quad \because \underbrace{W_N^{prk}}_{N\text{点DFT}} = W_{N/p}^{rk} = \underbrace{W_q^{rk}}_{q\text{点DFT}}$$

$$= \sum_{L=0}^{p-1} W_N^{Lk} \sum_{r=0}^{q-1} x(pr+L) W_q^{rk}$$

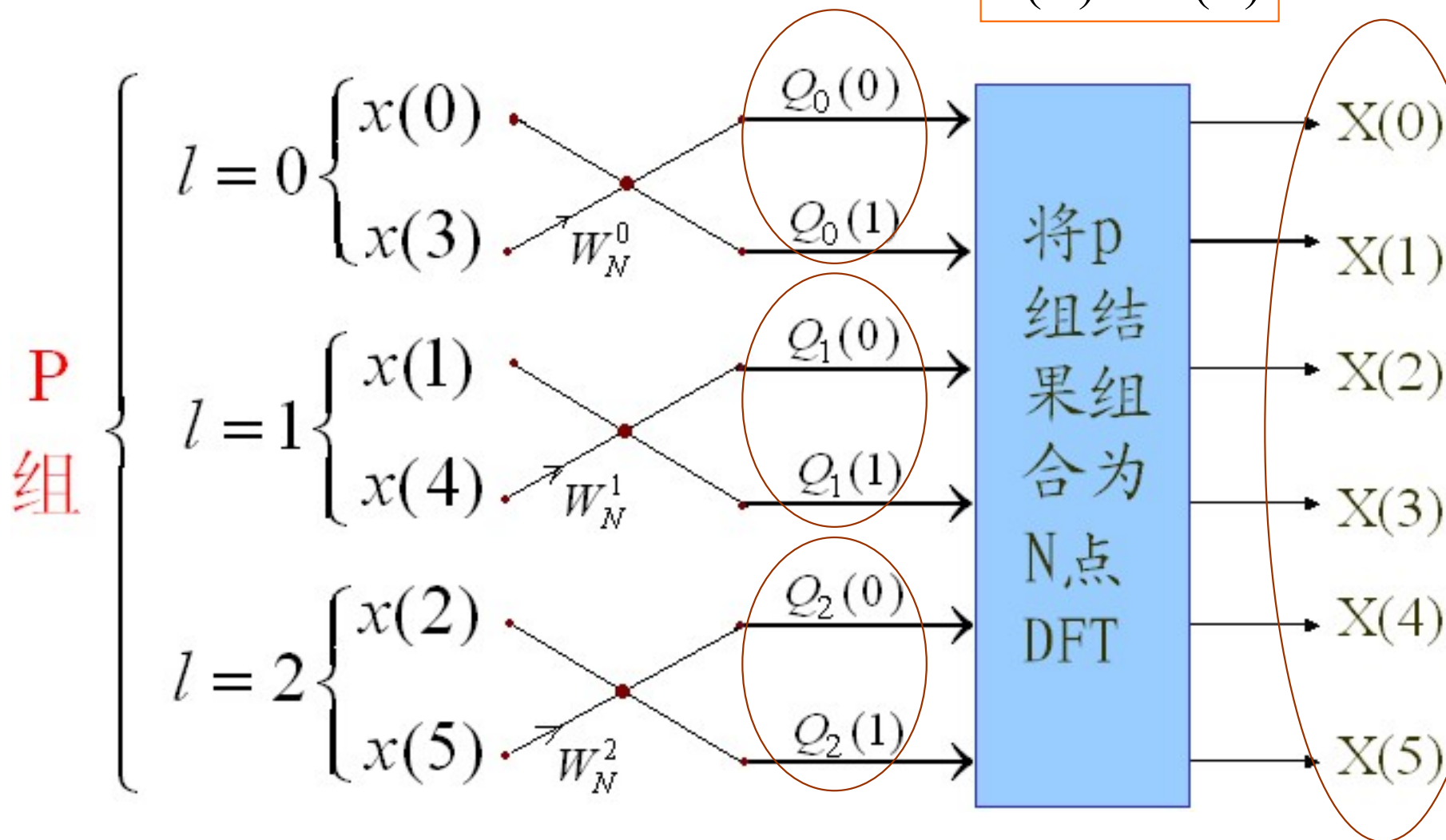
$$= \sum_{L=0}^{p-1} W_N^{Lk} \underbrace{Q_L(k)}_{q\text{点的DFT}}$$



例: $N=6$ $p=3$ $q=2$

P
组

$x(0)$	$x(3)$
$x(1)$	$x(4)$
$x(2)$	$x(5)$

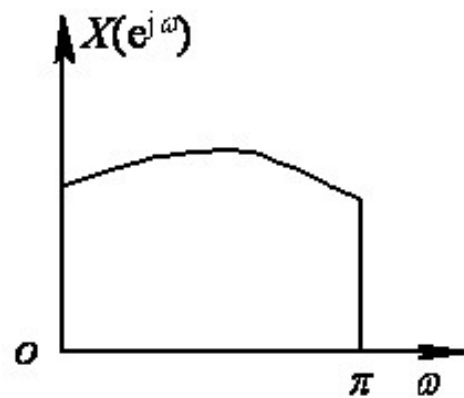
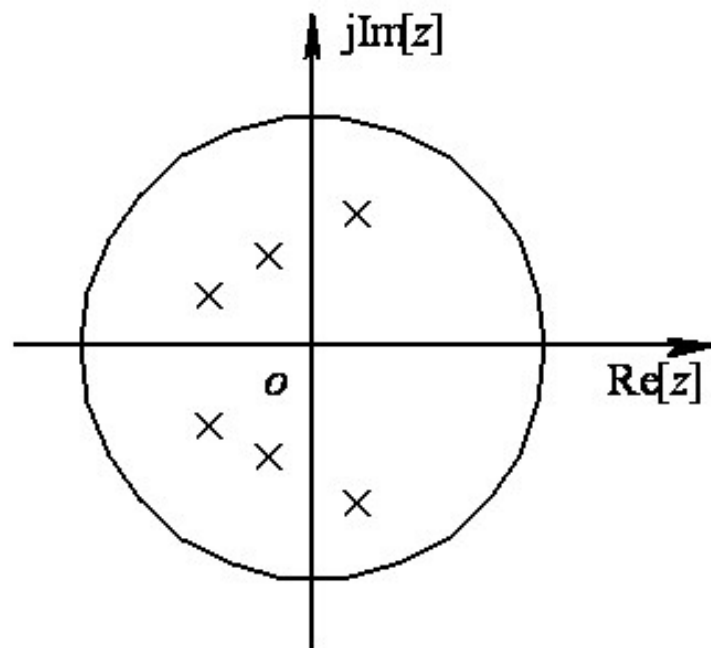




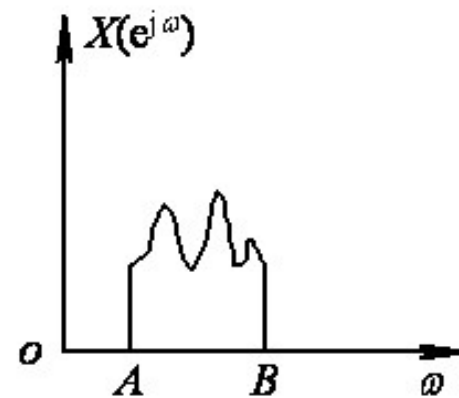
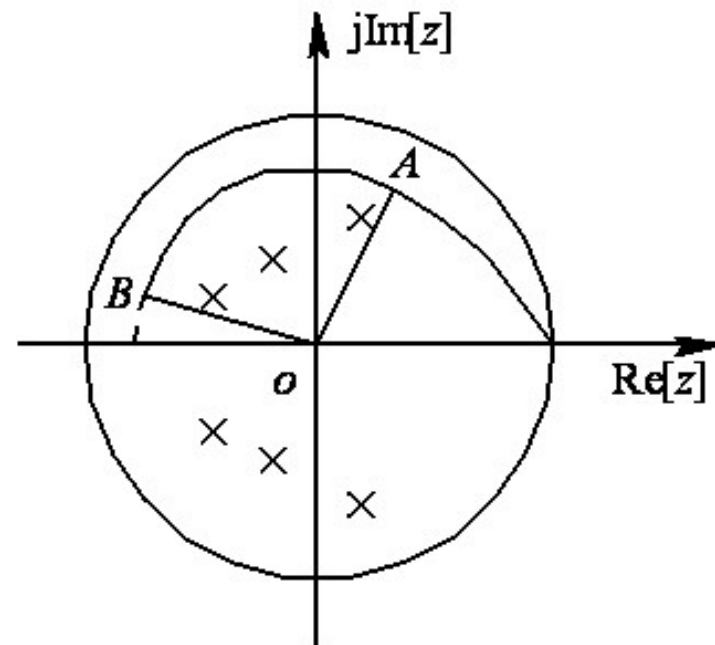
§ 4.6 Chirp-z变换

FFT算法可以很快计算出全部DFT值，即 Z 变换在单位圆上的采样值，但在很多场合，并非整个单位圆上的频谱都是有意义的，例如窄带信号过程，往往只需要对信号所在的一段频带进行分析，这时，希望采样能密集在这段频段内。

另外，有时也希望采样能不局限于单位圆上，如在语音信号处理中，往往需要知道其 z 变换的极点所在频率，若极点离单位圆较远，其单位圆上的频谱很平滑，就很难从中识别出极点所在频率，要是采样不是沿单位圆而是沿一条接近这些极点的弧线进行，所得的结果将会在极点所在频率上出现明显的尖峰，这样极点频率的测定就要准确的多。



(a) 沿单位圆采样



(b) 沿AB弧采样

图 4.19单位圆与非单位圆采样



4.6.1 算法基本原理

已知 $x(n)$ ($0 \leq n \leq N-1$) 是有限长序列, 其 Z 变换为

$$X(z) = \sum_{n=0}^{N-1} x(n)z^{-n}$$

为适应 z 可以沿 Z 平面更一般的路径取值, 故沿 Z 平面上的一段螺旋线作等分角的采样, z 的这些采样点 z_k 为

$$z_k = AW^{-k} \quad k=0, 1, \dots, M-1 \quad (4-22)$$

M 为所要分析的复频率的点数, 即采样点的总数, 不一定等于 N ; A 和 W 都是任意复数, 可表示为:



$$A = A_0 e^{j\theta_0} \quad (4-23)$$

$$W = W_0 e^{-j\phi_0} \quad (4-24)$$

将式 (4-23) 与式 (4-24) 代入式 (4-22)，可得

$$z_k = A_0 e^{j\theta_0} W_0^{-k} e^{jk\phi_0} = A_0 W_0^{-k} e^{j(\theta_0 + k\phi_0)}$$

因此有：

$$z_0 = A_0 e^{j\theta_0}$$

$$z_1 = A_0 W_0^{-1} e^{j(\theta_0 + \phi_0)}$$

$$z_k = A_0 W_0^{-k} e^{j(\theta_0 + k\phi_0)}$$

$$z_{M-1} = A_0 W_0^{-(M-1)} e^{j(\theta_0 + (M-1)\phi_0)}$$

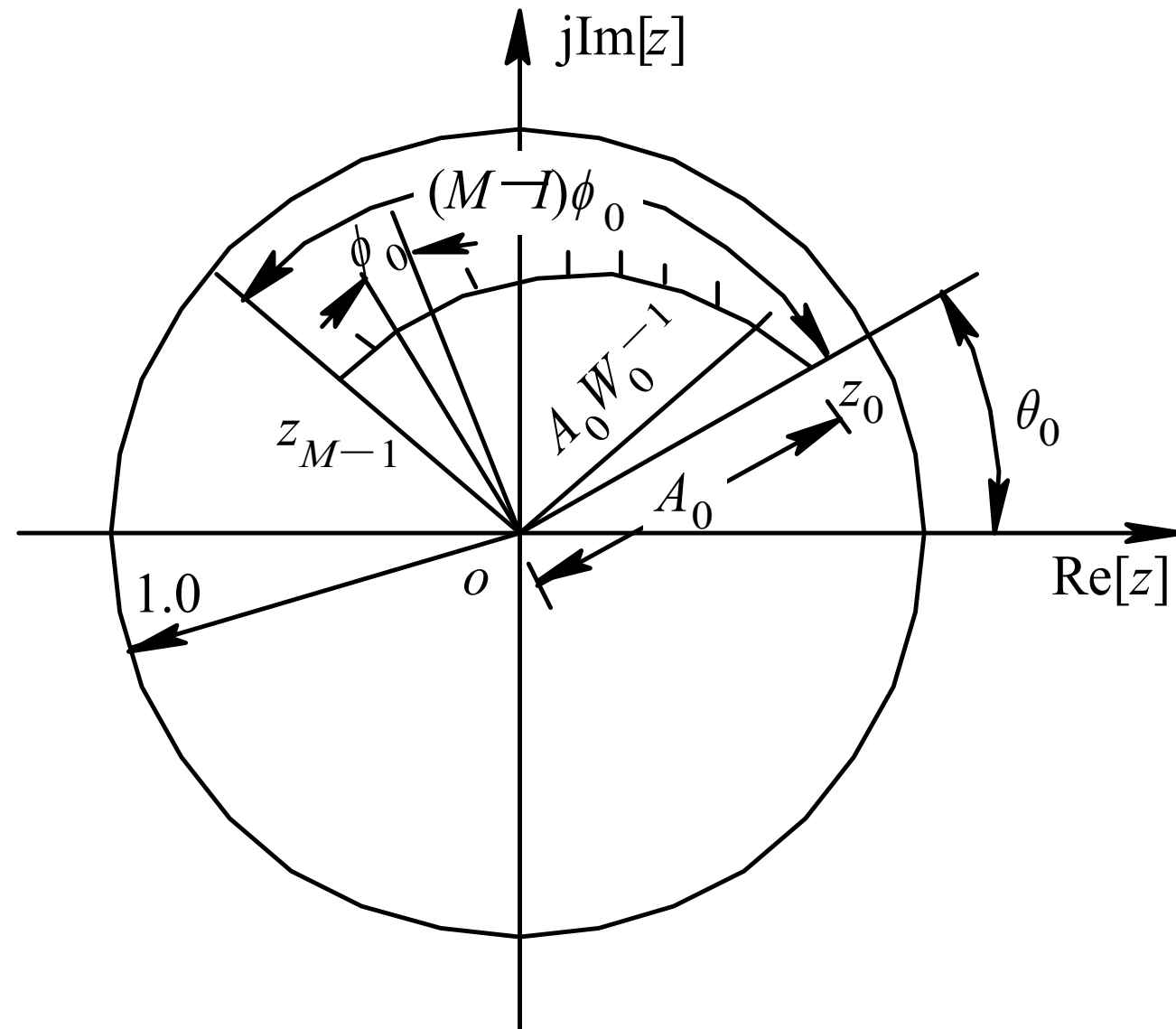


图 4-20 螺线采样



采样点在 z 平面的分布如图4-20所示。

由以上讨论和图4-20可以看出：

(1) A_0 表示起始采样点 z_0 的矢量半径

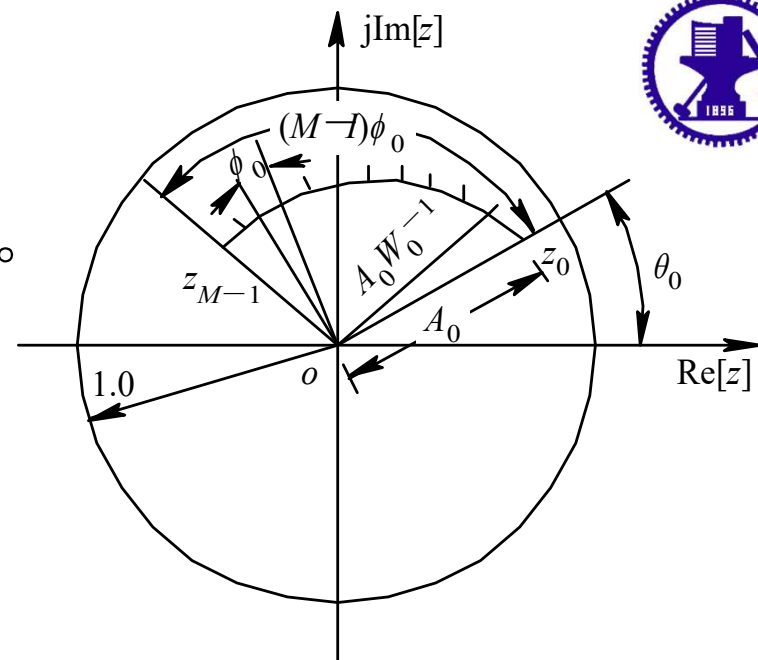
长度，通常 $A_0 \leq 1$ ；否则 z_0 将处于单位圆 $|z|=1$ 的外部；

(2) θ_0 表示起始采样点 z_0 的相角，它可以是正值或负值；

(3) φ_0 表示两相邻采样点之间的角度差。 φ_0 为正时，表示

z_k 的路径是逆时针旋转的； φ_0 为负时，表示 z_k 的路径是

顺时针旋转的；





(4) W_0 的大小表示螺线的伸展率。 $W_0 > 1$ 时, 随着 k 的增加螺线

内缩; $W_0 < 1$ 时, 则随 k 的增加螺线外伸; $W_0 = 1$ 时, 表示是半

径为 A_0 的一段圆弧。若又有 $A_0 = 1$, 则这段圆弧是单位圆的

一部分。

当 $M=N$, $A=A_0 e^{j\theta_0}=1$, $W=W_0 \cdot e^{-j\varphi_0} = -j \frac{2\pi}{N}$ ($W_0=1$, $\varphi_0=2\pi/N$) 这

一特殊情况时, 各 z_k 就均匀等间隔地分布在

单位圆上, 这就是求序列的DFT。

将式 (4-22) z_k 的代入Z变换表达式, 可得

